

Mestrado em Engenharia e Ciência de Dados

Visualização de Dados e Informação

Ficha tutorial de exercícios 7 – Grafos e Árvores

1. Para estudar o perfil de uma rede de utilizadores anonimizados do Facebook, recorreu-se ao *dataset Social circles: Facebook (SNAP)*, disponível em <https://snap.stanford.edu/data/egonets-Facebook.html>, do qual se extraiu o *"facebook_combined.txt"* que contém um conjunto de nós ligados, sendo que cada aresta ocupa uma linha e corresponde à ligação de um utilizador A a um utilizador B.
 - a. Começemos por carregar o ficheiro para obter algumas estatísticas básicas, bem como apresentar o aspeto do grafo (i.e., "teia" de ligações entre utilizadores) completo.

Código:

```
import networkx as nx
import matplotlib.pyplot as plt

G = nx.read_edgelist("facebook_combined.txt", nodetype=int)

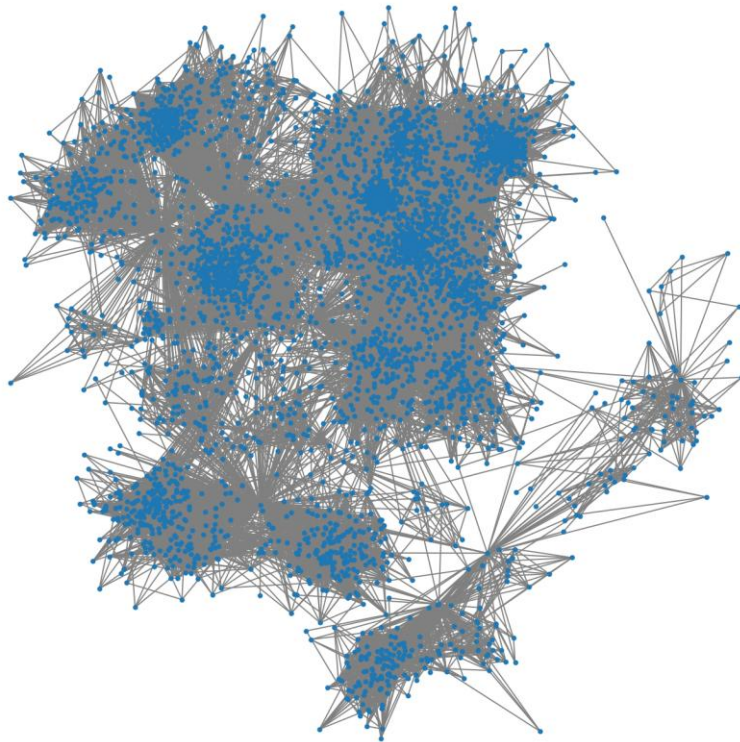
print("Nº de nós:", G.number_of_nodes())
print("Nº de arestas:", G.number_of_edges())

print("Grau médio:", sum(dict(G.degree()).values())/G.number_of_nodes())
print("Densidade:", nx.density(G))

plt.figure(figsize=(10, 10))
pos = nx.spring_layout(G, seed=42)
nx.draw(G, pos, node_size=10, edge_color="gray")
plt.title("Rede Social - Facebook (SNAP)")
plt.show()
```

Resultado esperado:

```
Nº de nós: 4039
Nº de arestas: 88234
Grau médio: 43.69101262688784
Densidade: 0.010819963503439287
```



- b. Vamos filtrar o grafo de forma que possamos trabalhar com os nós até ao ID 100.

Código:

```
import networkx as nx
import matplotlib.pyplot as plt

G = nx.read_edgelist("facebook_combined.txt", nodetype=int)

nos_filtrados = [n for n in G.nodes() if n <= 100]

G100 = G.subgraph(nos_filtrados).copy()

print("Nº de nós filtrados:", G100.number_of_nodes())
print("Nº de arestas filtradas:", G100.number_of_edges())

print("Grau médio:", sum(dict(G100.degree()).values())/G100.number_of_nodes())
print("Densidade:", nx.density(G100))

print("Nº de nós:", G.number_of_nodes())
print("Nº de arestas:", G.number_of_edges())

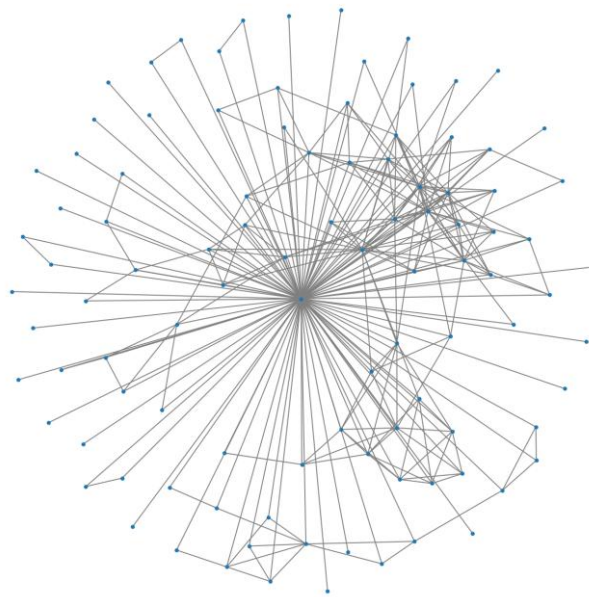
print("Grau médio:", sum(dict(G.degree()).values())/G.number_of_nodes())
print("Densidade:", nx.density(G))

plt.figure(figsize=(10, 10))
pos = nx.spring_layout(G100, seed=42)
nx.draw(G100, pos, node_size=10, edge_color="gray")
plt.title("Rede Social - Facebook (SNAP)")
plt.show()
```

Resultado esperado:

```
Nº de nós filtrados: 101  
Nº de arestas filtradas: 277  
Grau médio: 5.485148514851486  
Densidade: 0.05485148514851485  
Nº de nós: 4039  
Nº de arestas: 88234  
Grau médio: 43.69101262688784  
Densidade: 0.010819963503439287
```

Rede Social - Facebook (SNAP)



- c. Identifique as linhas de:
 - i. Carregamento do grafo pelo ficheiro;
 - ii. Filtragem pelo ID
 - iii. Apresentação de Estatísticas
 - iv. Apresentação visual
- d. Identifique os problemas de visualização face ao que foi falado na aula teórica, em termos de linhas orientadoras.
- e. Desenvolva uma função devolva todas as ligações de 2º grau de um dado nó do grafo.

Código:

```
def segundo_grau(G, no):
    if no not in G:
        raise ValueError(f"O nó {no} não existe no grafo.")

    viz1 = set(G.neighbors(no))

    viz2 = set()
    for v in viz1:
        viz2.update(G.neighbors(v))

    viz2.difference_update(viz1)
    viz2.discard(no)

    return list(viz2)
```

- f. Agora, vamos experimentar dimensionar os nós de acordo com o número de vizinhos aos quais estão associados (que é o mesmo que dizer o número de amigos).

Código:

```
import networkx as nx
import matplotlib.pyplot as plt

def tamanhos_por_grau(G, base=100, fator=50):
    graus = dict(G.degree()) # {nó: grau}
    sizes = [base + fator * graus[n] for n in G.nodes()]
    return sizes

G = nx.read_edgelist("facebook_combined.txt", nodetype=int)

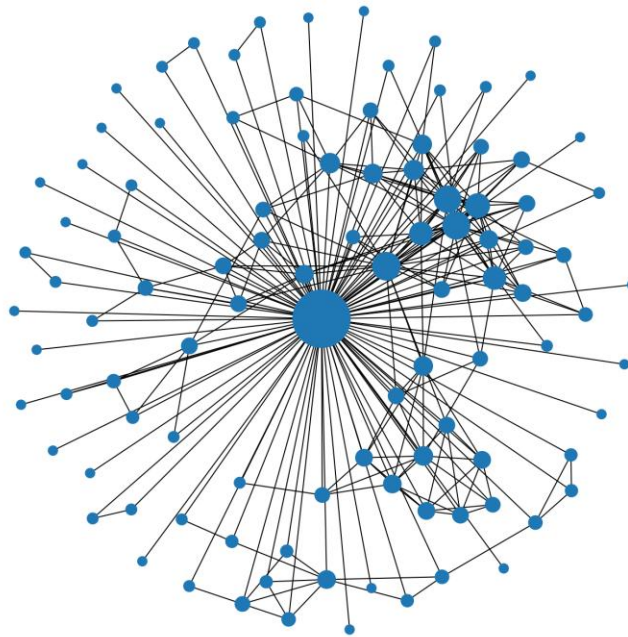
nos_filtrados = [n for n in G.nodes() if n <= 100]

G100 = G.subgraph(nos_filtrados).copy()

sizes = tamanhos_por_grau(G100, base=50, fator=30)

plt.figure(figsize=(10, 10))
pos = nx.spring_layout(G100, seed=42)
nx.draw(
    G100,
    pos,
    with_labels=False,
    node_size=sizes,
)
plt.title("Rede Social - Facebook (SNAP)")
plt.show()
```

Resultado esperado:



- g. Em seguida, será apresentado um conjunto de requisitos para uma abordagem de VDI experimental, que deverá resolver com recurso a fontes online, incluindo, serviços de IA generativo.
- i. Gere, de forma aleatória, um ficheiro de especificação que, por linha, contem um ID, um nome e um papel fictício, em que o ID, identifica um nó, o nome corresponde a alguém ligado à primeira liga de andebol e o papel identifica uma função (jogador, treinador, agente). Eis um exemplo:

0; Jorge Martins; jogador
1; Caetano Alfredo; jogador
2; Severino Couto; jogador
3; Júlio Cardoso; treinador
4; Esteves Castro; agente
...
...
...

- ii. Gere, também de forma aleatória, um ficheiro de ligação de nós seguindo o formato “facebook_combined.txt”, assegurando que os IDs constam no ficheiro gerado na alínea anterior.
 - iii. Apresente uma forma de visualização que, para além do tamanho do nó associado ao número de vizinhos (conforme já foi feito anteriormente), também é aplicado uma escala de cor considerando o número de agentes a que um jogador está ligado. Faça a escala de cor variar entre azul (nenhuma ligação a agentes) e vermelho (jogador com mais ligações a agentes na amostra).
 - iv. Reflita acerca do potencial de aplicação de grafos no presente contexto, enumerando e descrevendo um conjunto de cenários hipotéticos de uso.
2. Este exercício pretende ser um primeiro contacto com árvores geradas aleatoriamente e visualizadas em seguida. Para fazer este exercício, precisará instalar a biblioteca pygraphviz:

conda install conda-forge::pygraphviz

- i. Vamos criar uma função para gerar árvore, recorrendo ao seguinte código:

```
import networkx as nx
import matplotlib.pyplot as plt

def gerar_arvore(niveis=4, filhos=2):
    G = nx.DiGraph()
    root = 0
    G.add_node(root)

    nodos_atuais = [root]
    proximo_id = 1

    for nivel in range(1, niveis):
        novos_nodos = []
        for pai in nodos_atuais:
            for _ in range(filhos):
                filho = proximo_id
                proximo_id += 1
                G.add_edge(pai, filho)
                novos_nodos.append(filho)
            nodos_atuais = novos_nodos

    return G, root
```

- ii. Use a função anterior para gerar uma árvore com 4 níveis e 2 filhos por nó.

Código:

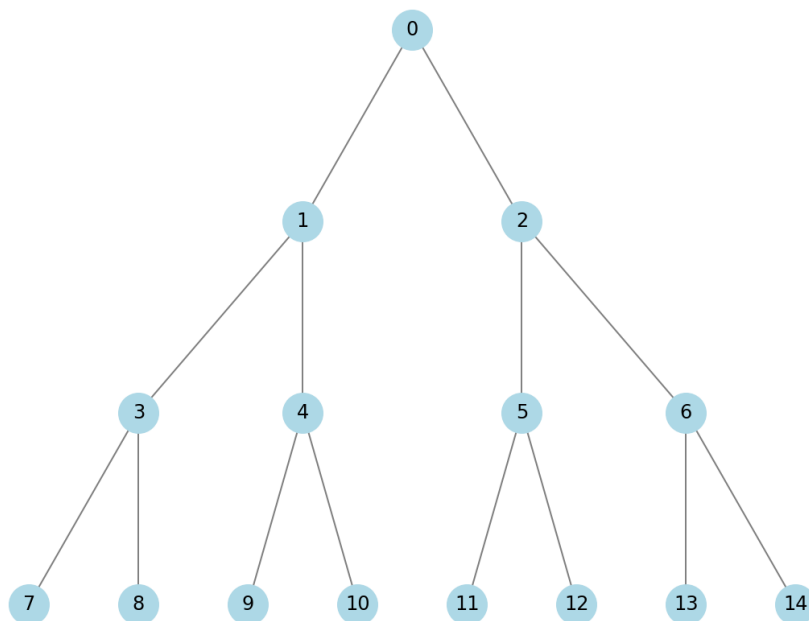
```
G, root = gerar_arvore(niveis=4, filhos=2)

pos = nx.nx_agraph.graphviz_layout(G, prog="dot") if hasattr(nx.nx_agraph, "graphviz_layout") \
    else nx.spring_layout(G, seed=42)

plt.figure(figsize=(8, 6))
nx.draw(
    G,
    pos,
    with_labels=True,
    arrows=False,
    node_size=600,
    node_color="lightblue",
    edge_color="gray"
)
plt.title("Árvore gerada automaticamente com 4 níveis")
plt.show()
```

Exemplo de saída possível:

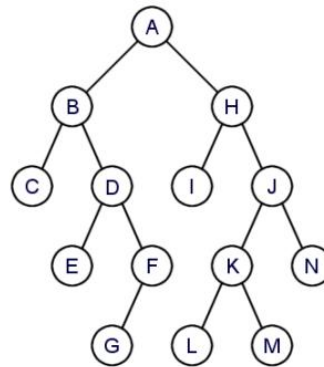
Árvore gerada automaticamente com 4 níveis



- iii. Agora, considere que se pretende testar as várias formas de travessia de uma árvore para determinar qual a maneira mais eficiente de chegar a um dado nó da árvore, admitindo que o “melhor” método visita menos nós para chegar ao destino.

Para mais informações sobre travessia de árvores binárias, consulte a primeira página da sebenta online disponível em <https://sweet.ua.pt/rosalia/cadeiras/TAD/TravessiasArvores.pdf>

- **Travessias de uma Árvore Binária:**



Pré-Ordem: 1º raiz;
2º sub-árvore esquerda;
3º sub-árvore direita.

{ A B C D E F G H I J K L M N }

Em-Ordem: 1º sub-árvore esquerda;
2º raiz;
3º sub-árvore direita.

{ C B E D G F A I H L K M J N }

Pos-Ordem: 1º sub-árvore esquerda;
2º sub-árvore direita;
3º raiz.

{ C E G F D B I L M K N J H A }

Função para fazer a travessia de uma árvore, contemplando os vários métodos:


```
def travessia_arvore(G, inicio=0, destino=None, modo="pre"):
    visitados = []

    # Função recursiva interna
    def dfs(n):
        filhos = list(G.successors(n))

        if destino is not None and n == destino:
            visitados.append(n)
            return True

        # PRE-ORDEN
        if modo == "pre":
            visitados.append(n)

        # ESQ
        if len(filhos) >= 1:
            if dfs(filhos[0]):
                return True

        # IN-ORDEN
        if modo == "in":
            visitados.append(n)

        # DIR
        if len(filhos) >= 2:
            if dfs(filhos[1]):
                return True

        # POST-ORDEN
        if modo == "post":
            visitados.append(n)

        return False

    dfs(inicio)

    return visitados, len(visitados)
```

Conjunto de chamadas de exemplo:

```
print(travessia_arvore(G, destino=8, modo="pre"))
print(travessia_arvore(G, destino=8, modo="in"))
print(travessia_arvore(G, destino=8, modo="post"))
```

Resultado:

```
([0, 1, 3, 7, 8], 5)
([7, 3, 8], 3)
([7, 8], 2)
```